

# Computable and uncomputable functions

## Lecture 1

### 1 Alphabets and words

At the low level, information inside computers is represented by 0s and 1s. Every program takes as input a finite sequence of bits and returns another such sequence, and all computation as we know it, in the end, is manipulation of bit-strings. Our formalisation begins here.

**Definition 1** (Alphabets and words). An **alphabet**  $\Sigma$  is a non-empty finite set of symbols. A **word** (or **string**) over  $\Sigma$  is a finite sequence of symbols  $w = a_1 a_2 \dots a_n$ , where each  $a_i \in \Sigma$ . The unique sequence of length zero is the **empty word**, denoted  $\varepsilon$ . The set of all finite words over  $\Sigma$  is denoted  $\Sigma^*$ .

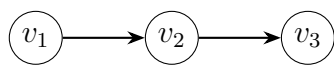
Given two words  $u = a_1 \dots a_n$  and  $v = b_1 \dots b_m$ , their **concatenation** is  $uv = a_1 \dots a_n b_1 \dots b_m$ . Concatenation is associative:  $(uv)w = u(vw)$ , and  $\varepsilon$  is the two-sided identity:  $u\varepsilon = \varepsilon u = u$ .<sup>1</sup>

**Binary encoding.** The ‘canonical’ alphabet for computers is the *binary alphabet*  $\{0, 1\}$ . Words in any other alphabet  $\Sigma$  can be *encoded* by binary words as follows. We fix a length  $k$  such that  $2^k \geq |\Sigma|$ , and assign a distinct  $k$ -bit binary word to each symbol  $a_i$ . For example, if  $\Sigma = \{a, b, c\}$ , we can choose  $k = 2$  and map

$$a \mapsto 00, \quad b \mapsto 01, \quad c \mapsto 10.$$

A word over  $\Sigma$  is then encoded by replacing each symbol with its code: for instance,  $cab$  becomes 100001. Since every symbol uses exactly 2 bits (or, in general,  $k$ ), decoding is unambiguous: we simply read the string two bits at a time.<sup>2</sup>

Other finite objects that programs work with are encoded similarly. For example, a directed graph  $G$  on vertices  $\{v_1, \dots, v_n\}$  can be represented by its  $n \times n$  adjacency matrix  $A$ , where  $A_{i,j} = 1$  if there is an edge  $v_i \rightarrow v_j$  and 0 otherwise. Flattening this matrix row by row yields a binary word of length  $n^2$ . For instance, the graph on 3 vertices with edges  $v_1 \rightarrow v_2$  and  $v_2 \rightarrow v_3$  and its corresponding adjacency matrix are:


 $A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix},$

which is encoded by the word 010001000.

<sup>1</sup>If you like algebra, you may note that  $\Sigma^*$  with concatenation and  $\varepsilon$  is thus the free monoid generated by  $\Sigma$ .

<sup>2</sup>In ASCII, a standard encoding in real computers, each character is an 8-bit byte.

## 2 Computable functions

Fix your favourite general-purpose programming language (Python, C++, Haskell, Java...). Consider programs that take one input word and produce one output word over some alphabet  $\Sigma$ . As explained above, multiple arguments, numbers, graphs, or structured data can always be encoded into a single word in  $\Sigma^*$  (and decoded from the output).

Thus, each program  $\pi$  naturally computes a (partial!) function  $f_\pi: \Sigma^* \rightarrow \Sigma^*$ .<sup>3</sup> This is how it is defined.

**Definition 2** (Function computed by a program  $\pi$ ). *Given  $u \in \Sigma^*$ , run  $\pi$  on  $u$ .*

- *If  $\pi$  halts (i.e., terminates, concludes the execution), then  $f_\pi(u) = v$ , where  $v \in \Sigma^*$  is the word it outputs.*
- *If  $\pi$  does not halt, then  $f_\pi(u)$  is undefined.*

In real life, programs may also crash with an error. In theory, we treat crashes as equivalent to non-halting.

**Definition 3** (Computable function). *A partial function  $f: \Sigma^* \rightarrow \Sigma^*$  is called **computable** if there exists a program  $\pi$  such that  $f = f_\pi$ .*

Note that when  $f: \Sigma^* \rightarrow \Sigma^*$  is **total**, i.e., defined on all words in  $\Sigma^*$ , the corresponding program  $\pi$  must halt on every input.

Let us consider some examples of computable functions.

**Example 4.** The following functions are all computable:

- *reverse*: given  $w \in \Sigma^*$ , outputs its reverse (e.g.,  $\text{reverse}(abc) = cba$ );
- *convert*: interprets  $w \in \{0, 1, \dots, 9\}^*$  as an integer  $n$ , outputs a word in  $\{0, 1, 2, 3\}$ —the same number, but in 4-ary notation;
- *double*: interprets  $w$  as an integer  $n$ , outputs  $2n$ ;
- *multiply*: given a pair of integers  $\langle n, k \rangle$ , outputs their product  $n \cdot k$ ;
- *shortestPath*: given a directed graph and two vertices  $\langle G, s, t \rangle$ , outputs the length of the shortest path from  $s$  to  $t$  (undefined if no path exists);
- *deanon*: given  $w$ , if  $w$  encodes the real name of Banksy,  $\text{deanon}(w)$  encodes the real name of Satoshi Nakamoto; for all other  $w$ , *deanon* is undefined.

---

<sup>3</sup>Note the arrow! It is  $\rightarrow$ , not  $\mapsto$ . This is how we denote that the function is *partial*.

### 3 Uncomputable functions

Are there any functions that are not computable? Yes, and it is easy to prove: there are only countably many programs, but uncountably many functions on  $\{0,1\}^*$ . Let us see this argument in detail.

Recall that a set  $S$  is called *countable* if its elements can be enumerated by natural numbers. The set  $\{0,1\}^*$  is, of course, countable—here is a possible enumeration of words in  $\{0,1\}^*$ :

$$\begin{aligned}
 w_0 &= \varepsilon \\
 w_1 &= 0 \\
 w_2 &= 1 \\
 w_3 &= 00 \\
 w_4 &= 01 \\
 w_5 &= 10 \\
 w_6 &= 11 \\
 w_7 &= 000 \\
 &\dots
 \end{aligned} \tag{1}$$

That is, we list the words by increasing length, and order words of the same length lexicographically. Similarly,  $\Sigma^*$  is countable for any alphabet  $\Sigma$ .

**Programs are words themselves.** Indeed, programs are just sequences of symbols that we write, and thus can be seen as words. In fact, the converse is also true: we can try to execute any word as if it was a program and see what happens—a crash would just mean non-halting. So every program is a word and every word encodes some (maybe nonsensical) program. The set of all programs, and hence of all computable functions, is therefore countable.

In contrast, the set of all partial functions  $\{0,1\}^* \rightarrow \{0,1\}^*$  is uncountable. In fact, this is already true for the set of all functions  $\{0,1\}^* \rightarrow \{0,1\}$ .

**Proposition 5** (Cantor, 1891). *The set of all functions  $\{0,1\}^* \rightarrow \{0,1\}$  is uncountable.*

*Proof.* For clarity of notation, we use the correspondence (1) and consider the set of functions  $\mathbb{N} \rightarrow \{0,1\}$ . Assume for contradiction that this set is countable. Then we can enumerate all functions in this set:  $f_0, f_1, f_2, \dots$ . Each function  $f_i$  is defined by its values on each  $k \in \mathbb{N}$ . Let us write them out:

|                   |                   |                   |                   |         |
|-------------------|-------------------|-------------------|-------------------|---------|
| $\mathbf{f_0(0)}$ | $f_0(1)$          | $f_0(2)$          | $f_0(3)$          | $\dots$ |
| $f_1(0)$          | $\mathbf{f_1(1)}$ | $f_1(2)$          | $f_1(3)$          | $\dots$ |
| $f_2(0)$          | $f_2(1)$          | $\mathbf{f_2(2)}$ | $f_2(3)$          | $\dots$ |
| $f_3(0)$          | $f_3(1)$          | $f_3(2)$          | $\mathbf{f_3(3)}$ | $\dots$ |
| $\dots$           |                   |                   |                   |         |

Consider the diagonal of this infinite table, in bold, and the function:

$$d(n) = \begin{cases} 1, & \text{if } f_n(n) = 0, \\ 0, & \text{if } f_n(n) = 1. \end{cases}$$

Then,  $d$  is a function  $\mathbb{N} \rightarrow \{0, 1\}$ , so it must appear in the above list  $f_0, f_1, f_2, \dots$  of all such functions. However, by construction,  $d$  disagrees with  $f_0$  on 0, with  $f_1$  on 1, and, in general, with  $f_n$  on  $n$ . A contradiction.  $\square$

**Corollary 6.** *There exist uncomputable functions from  $\{0, 1\}^*$  to  $\{0, 1\}$ .*

In fact, there are uncountably many.

And so what? No concrete example was given so far. Could it be that all *useful* functions that we may ever want to compute are countable? Unfortunately, not. Consider the following very useful *halting function*:

$$h(\pi, w) = \begin{cases} 1, & \text{if } \pi \text{ halts on input } w, \\ 0, & \text{if } \pi \text{ loops forever on input } w. \end{cases}$$

It takes two arguments: a program  $\pi$  and a word  $w$ , but of course the comma can be treated as a special symbol in a word, so  $h$  is a function  $\Sigma^* \rightarrow \Sigma^*$  just like those we considered before. Note also that it is a total function.

**Theorem 7** (Turing, 1936). *The halting predicate is (alas!) uncomputable.*

*Proof.* Recall that we have an enumeration of words in  $\Sigma^*$ :  $w_0, w_1, w_2, \dots$ , each of which can be seen as a program:  $\pi_0, \pi_1, \pi_2, \dots$ . Each  $\pi_i$  can (potentially) be run on each  $w_j$  (if  $i = j$ , we run  $\pi_i$  on its own code!)

We can imagine a table of results for the halting predicate:

|          | $w_0$                             | $w_1$                             | $w_2$                             | $w_3$                             | $\dots$  |
|----------|-----------------------------------|-----------------------------------|-----------------------------------|-----------------------------------|----------|
| $\pi_0$  | <b><math>h(\pi_0, w_0)</math></b> | $h(\pi_0, w_1)$                   | $h(\pi_0, w_2)$                   | $h(\pi_0, w_3)$                   | $\dots$  |
| $\pi_1$  | $h(\pi_1, w_0)$                   | <b><math>h(\pi_1, w_1)</math></b> | $h(\pi_1, w_2)$                   | $h(\pi_1, w_3)$                   | $\dots$  |
| $\pi_2$  | $h(\pi_2, w_0)$                   | $h(\pi_2, w_1)$                   | <b><math>h(\pi_2, w_2)</math></b> | $h(\pi_2, w_3)$                   | $\dots$  |
| $\pi_3$  | $h(\pi_3, w_0)$                   | $h(\pi_3, w_1)$                   | $h(\pi_3, w_2)$                   | <b><math>h(\pi_3, w_3)</math></b> | $\dots$  |
| $\vdots$ | $\vdots$                          | $\vdots$                          | $\vdots$                          | $\vdots$                          | $\ddots$ |

We are going to mess with the diagonal again! Suppose for contradiction that  $h$  is computed by some program  $\eta$ . We define the following *diagonal* program  $\delta$ :

```
def  $\delta(w)$ :
    if  $\eta(w, w) == 1$ :
        while True: pass    # loop forever
    else:
        return 0            # halt immediately
```

Since  $\delta$  is a valid program, we could ask what happens when  $\delta$  is run on its own code. If  $\delta$  halts when run on the word encoding  $\delta$ , then  $\eta(\delta, \delta)$  must return 1, and so  $\delta$ , according to its own code, should actually loop forever. If it does not halt,  $\eta(\delta, \delta)$  returns 0, and then  $\delta$  should halt. A contradiction.  $\square$